

Voice Based Concept Understanding Analyser:

AI-Powered Speech Evaluation & Concept Scoring Platform

Project Description:

The Voice Based Concept Understanding Analyser (VBCUA) harnesses Artificial Intelligence to evaluate a student's conceptual understanding by analyzing their spoken answers. The platform includes an Upload Audio module, where a student's voice recording and the corresponding expected answer are submitted, an AI Analysis engine that converts speech to text and evaluates the response, and a Dashboard and Report module that presents the score, similarity percentage, grade, transcript, and detailed AI feedback.

Utilizing OpenAI's Whisper AI for speech-to-text transcription and Google's Gemini AI for semantic evaluation, VBCUA processes a student's uploaded audio to deliver an accurate, fast, and consistent assessment of conceptual understanding. Built with Streamlit and backed by SQLite for analysis history and ReportLab for automated PDF report generation, the platform ensures a seamless, teacher-friendly experience. With a clean multi-page interface and downloadable reports, VBCUA empowers educators, trainers, and institutions to evaluate spoken understanding at scale with confidence.

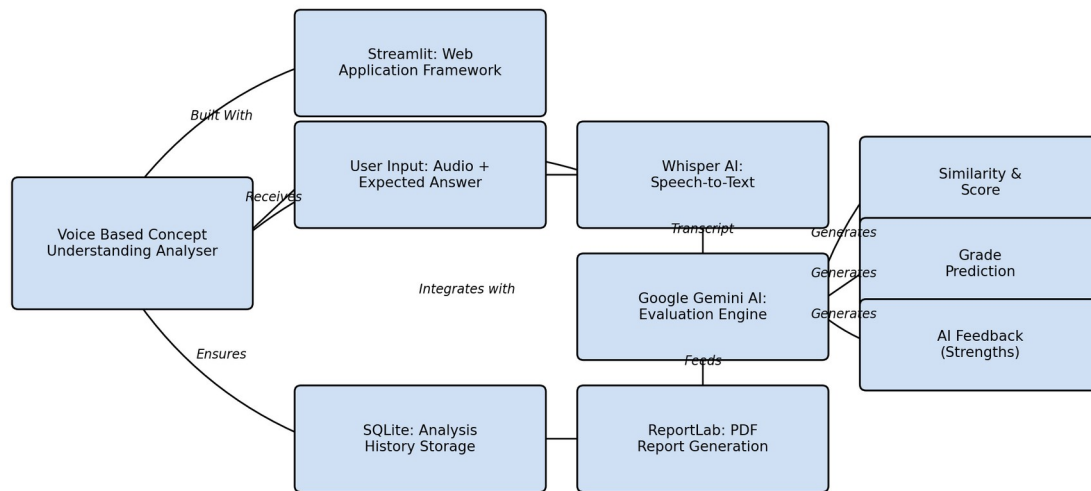
Scenarios

Scenario 1: A teacher uploads a student's voice recording answering "What is Artificial Intelligence?" along with the expected model answer. The system transcribes the audio using Whisper AI, compares it against the expected answer using semantic similarity, and generates a similarity percentage, a numeric score out of 100, and a letter grade within seconds.

Scenario 2: A student preparing for an oral exam records their own explanation of a concept and uploads it for self-assessment. CaptionAI-style AI feedback highlights the strengths of the answer and the specific gaps in understanding, helping the student revise before the actual exam.

Scenario 3: An instructor reviewing an entire batch of submissions opens the Dashboard to see the performance summary for each student and the Report page to download an AI-generated PDF report per student, building a complete, exportable record of each learner's conceptual grasp over time.

Technical Architecture:



Description:

VBCUA utilizes a modular architecture to deliver accurate, AI-driven speech evaluation. The Streamlit-based frontend provides a responsive multi-page interface for uploading audio and viewing results, while the backend orchestrates transcription and evaluation. Whisper AI converts the uploaded audio into text, and Google Gemini AI compares that transcript against the expected answer to produce a similarity score, numeric score, grade, and structured feedback. Results are persisted in a SQLite database for historical tracking, and ReportLab compiles the final results into a downloadable PDF report.

(Note: The application uses a lightweight SQLite database to persist each student's transcript, score, similarity percentage, and grade, enabling the Student Analysis History feature referenced on the Home page.)

Pre-requisites:

- **Python Programming Proficiency:** Python Programming Proficiency: [Python Documentation](#)
- **Streamlit Framework Knowledge:** Streamlit Framework Knowledge: [Streamlit Documentation](#)
- **OpenAI Whisper Familiarity:** OpenAI Whisper Familiarity: [Whisper GitHub Documentation](#)
- **Google Gemini API Familiarity:** Google Gemini API Familiarity: [Google AI Studio Documentation](#)
- **SQLite & Basic SQL Knowledge:** SQLite & Basic SQL Knowledge: [SQLite Documentation](#)
- **ReportLab for PDF Generation:** ReportLab for PDF Generation: [ReportLab User Guide](#)
- **Version Control with Git:** Version Control with Git: [Git Documentation](#)
- **Development Environment Setup:** Development Environment Setup: [Python venv Guide](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Activity 1.1: Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Activity 1.2: Research and select the appropriate speech-to-text and generative AI models for concept evaluation.
- **Activity 1.3:** Activity 1.3: Define the architecture of the application, detailing interactions between the frontend, transcription engine, and AI evaluation engine.
- **Activity 1.4:** Activity 1.4: Set up the development environment, installing necessary libraries and dependencies for Streamlit, Whisper, and Gemini.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Activity 2.1: Develop the core functionalities: Audio Transcription, Similarity Analysis, Score Generation, and Grade Prediction.
- **Activity 2.2:** Activity 2.2: Implement the Streamlit backend to manage page navigation and input processing, ensuring smooth AI interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Activity 3.1: Write the main application logic across Home.py and the page modules, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Activity 4.1: Design and develop the user interface using Streamlit and custom CSS, ensuring a clean, dark-themed, and intuitive layout.
- **Activity 4.2:** Activity 4.2: Create dynamic result rendering to display the transcript, score, similarity, and grade based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Activity 5.1: Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Activity 5.2: Test the application locally across multiple sample recordings to verify transcription and scoring accuracy.
- **Activity 5.3:** Activity 5.3: Deploy the application publicly to expose the local Streamlit server over a secure, shareable URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate speech-to-text and generative AI models for our concept-understanding evaluation needs. This involves researching the capabilities and performance of Whisper AI variants and Google Gemini AI, ensuring that the chosen models align well with the application's objectives of transcribing spoken answers and evaluating conceptual accuracy.

Activity 1.1: Generate a Google Gemini API Key

Before the application can communicate with the Gemini LLM, you need a valid Gemini API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Google AI Studio Account:** Step 1 — Create a Google AI Studio Account: Visit <https://aistudio.google.com> and sign in using your Google account.
- **Step 2 — Navigate to API Keys:** Step 2 — Navigate to API Keys: Click on "Get API Key" from the left-hand navigation menu, or go directly to the API Keys page.
- **Step 3 — Create a New API Key:** Step 3 — Create a New API Key: Click the "Create API Key" button and select a Google Cloud project to associate the key with.
- **Step 4 — Copy the API Key:** Step 4 — Copy the API Key: Copy the generated key immediately and store it in a safe place, as it will only be shown once.
- **Step 5 — Add the Key to Your Project:** Step 5 — Add the Key to Your Project: Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below.

```
GEMINI_API_KEY=your_copied_api_key_here
```

- **Step 6 — Verify the Key is Loaded:** Step 6 — Verify the Key is Loaded: The app loads this key automatically at startup using python-dotenv.

```
from dotenv import load_dotenv
import google.generativeai as genai
load_dotenv()
genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository.

```
.env
```

Activity 1.2: Research and Select the Appropriate AI Models

Here are the things that needed to be researched to select the best models for the project:

- **Understand the Project Requirements:** Understand the Project Requirements: Review the specific needs of the VBCUA application, focusing on accurate speech transcription and reliable semantic evaluation of spoken answers against expected answers.
- **Explore Whisper's Model Variants:** Explore Whisper's Model Variants: Compare the tiny, base, small, and medium Whisper models based on transcription accuracy, speed, and resource requirements for classroom-scale audio clips.
- **Explore Gemini's Model Documentation:** Explore Gemini's Model Documentation: Visit the Google AI documentation to examine available Gemini models, including their context windows, reasoning quality, and response latency.
- **Select the Optimal Models:** Select the Optimal Models: Choose the Whisper "base" model for a strong balance of transcription accuracy and speed, and gemini-1.5-flash for fast, cost-effective semantic evaluation and structured JSON feedback.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Draft an Architectural Diagram: Create a visual representation of the application architecture, including components like the Streamlit frontend, Whisper transcription engine, and Gemini evaluation engine.
- **Detail Frontend Functionality:** Detail Frontend Functionality: Outline how users interact with the application uploading a student's audio file, entering the expected answer, and navigating between Home, Upload Audio, AI Analysis, Dashboard, and Report pages.
- **Outline Backend Responsibilities:** Outline Backend Responsibilities: Specify how the backend transcribes the uploaded audio, validates and sanitizes the expected answer input, builds a structured evaluation prompt, and communicates with the Gemini API.
- **Describe AI Integration Points:** Describe AI Integration Points: Define how the backend constructs prompts using the transcript and expected answer, calls the Gemini API, parses the JSON response via `extract_json()`, and stores structured results in SQLite for the Dashboard and Report pages.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Install Python and Pip: Ensure Python 3.9+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Streamlit:** Install Streamlit: Use pip to install Streamlit.

```
(myenv) D:\projects> pip install streamlit==1.38.0
```

- **Install Whisper AI:** Install Whisper AI: Install OpenAI's Whisper library and its FFmpeg dependency for audio decoding.

```
(myenv) D:\projects> pip install openai-whisper==20231117
```

- **Install the Google Generative AI SDK:** Install the Google Generative AI SDK: Install the Gemini Python library to interact with the Gemini API.

```
(myenv) D:\projects> pip install google-generativeai==0.8.3
```

- **Install python-dotenv, ReportLab, and Supporting Libraries:** Install python-dotenv, ReportLab, and Supporting Libraries: Install dotenv support for secure API key management, ReportLab for PDF generation, and scikit-learn for similarity scoring.

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
(myenv) D:\projects> pip install reportlab==4.2.5
(myenv) D:\projects> pip install scikit-learn==1.5.2
```

- **Configure the API Key:** Configure the API Key: Create a .env file in the project root and add your Gemini API key.

```
GEMINI_API_KEY=your_gemini_api_key_here
```

- **Set Up Application Structure:** Set Up Application Structure: Create the project directory structure including folders for pages, assets, and main application files (Home.py, database.py, utils.py, requirements.txt).

```
voice-concept-analyser/
├── pages/
│   ├── 1_Upload_Audio.py
│   ├── 2_AI_Analysis.py
│   ├── 3_Dashboard.py
│   └── 4_Report.py
├── assets/
│   └── style.css
├── .env
└── .gitignore
```

```
|— Home.py  
|— database.py  
|— utils.py  
|— requirements.txt
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Analysis Features

Implement the four core evaluation features — Transcription, Similarity Analysis, Score Generation, and Grade Prediction — as a unified AI response driven by a single structured evaluation prompt.

- **Speech-to-Text Transcription:** Uses Whisper AI to convert the uploaded student audio into an accurate text transcript, handling background noise and varied accents.
- **Similarity Analysis:** Computes a semantic similarity percentage between the generated transcript and the teacher-provided expected answer using Gemini's reasoning alongside TF-IDF cosine similarity as a numerical cross-check.
- **Score & Grade Generation:** Produces a numeric score out of 100 and maps it to a letter grade (A–F) based on configurable thresholds.
- **AI Feedback Generator:** Returns structured feedback listing the strengths of the student's answer and the specific concepts that were missed or misunderstood.

The numeric score returned by Gemini is mapped to a letter grade using the following configurable thresholds:

Score Range	Grade	Interpretation
90 – 100	A	Excellent conceptual understanding
75 – 89	B	Strong understanding with minor gaps
60 – 74	C	Adequate understanding, some gaps
40 – 59	D	Weak understanding, needs revision
Below 40	F	Insufficient understanding demonstrated

Activity 2.2: Implement the Streamlit Backend

Define Pages in Streamlit:

- Set up a multi-page Streamlit app using the pages/ directory convention for Upload Audio, AI Analysis, Dashboard, and Report.

Process User Input:

- Create a file uploader and text area in Upload Audio for users to submit their audio recording and expected answer.
- Use `st.session_state` to persist the uploaded audio, expected answer, and analysis results across pages.
- Validate that both the audio file and the expected answer are provided before allowing analysis to proceed.

Integrate AI Calls:

- Within the AI Analysis page, call the `transcribe_audio()` function to convert speech to text using Whisper.
- Submit the transcript and expected answer to the Gemini API using `genai.GenerativeModel().generate_content()` with the `gemini-1.5-flash` model.
- Parse and validate the JSON response using `extract_json()`, which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of Home.py and the page modules, which serve as the backbone of the VBCUA application. In this milestone, you'll set up each feature's functionality through Streamlit pages, establish reliable transcription and evaluation logic, and conduct testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic

Define the Home Page in Home.py:

```
import streamlit as st

st.set_page_config(page_title="Voice Based Concept Understanding Analyser",
                    page_icon="🧠", layout="wide")

with open("assets/style.css") as f:
    st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)

st.title("🧠 Voice Based Concept Understanding Analyser")
st.header("🌸 Welcome")
st.write("This application uses Artificial Intelligence to evaluate a "
        "student's conceptual understanding by analyzing voice recordings.")
```

Define the Upload Audio Page in pages/1_Upload_Audio.py:

```
import streamlit as st

st.title("🧠 Upload Student Audio")

audio_file = st.file_uploader("Choose Audio File", type=["mp3", "wav", "m4a"])
expected_answer = st.text_area("Enter Expected Answer", height=160)

if audio_file:
    st.audio(audio_file)

if st.button("💾 Save Input"):
    if audio_file and expected_answer.strip():
        st.session_state["audio_file"] = audio_file
        st.session_state["expected_answer"] = expected_answer
        st.success(f"Uploaded: {audio_file.name}")
        st.success("Audio and Expected Answer saved successfully.")
    else:
        st.error("Please upload audio and enter the expected answer.")
```

Transcribe the Audio with Whisper AI (utils.py):

```
import whisper

model = whisper.load_model("base")

def transcribe_audio(audio_path: str) -> str:
    """Transcribe a student's spoken answer into text using Whisper AI."""
    result = model.transcribe(audio_path)
    return result["text"].strip()
```

Evaluate the Transcript with Gemini AI (utils.py):

```
import google.generativeai as genai

gemini_model = genai.GenerativeModel("gemini-1.5-flash")

def evaluate_answer(transcript: str, expected_answer: str) -> dict:
    """Compare the transcript against the expected answer and return
    similarity, score, grade, and structured feedback as JSON."""
    prompt = f"""You are an expert academic evaluator. Compare the
    student transcript to the expected answer and return STRICT JSON
    with keys: similarity (0-100), score (0-100), grade (A-F),
    strengths (list of strings), gaps (list of strings).

    Expected Answer: {expected_answer}
    Student Transcript: {transcript}"""

    response = gemini_model.generate_content(prompt)
    return extract_json(response.text)
```

Parse the Gemini JSON Response (utils.py):

```
import json, re

def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    match = re.search(r'```(?:json)?\s*([\s\S]*?)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r'\{([\s\S]*?)\}', text)
    if match:
        try:
            return json.loads(match.group(0))
```

```
except json.JSONDecodeError:  
    pass  
raise ValueError("Could not extract valid JSON from response")
```

Define the AI Analysis Page in pages/2_AI_Analysis.py:

```
import streamlit as st
from utils import transcribe_audio, evaluate_answer
from database import save_result

st.title("🧠 AI Analysis")

if "audio_file" not in st.session_state:
    st.warning("Please upload audio first on the Upload Audio page.")
else:
    if st.button("▶ Run AI Analysis"):
        with st.spinner("Transcribing audio..."):
            transcript = transcribe_audio(st.session_state["audio_file"])
        with st.spinner("Evaluating with Gemini AI..."):
            result = evaluate_answer(transcript,
                                    st.session_state["expected_answer"])
        st.session_state["transcript"] = transcript
        st.session_state["result"] = result
        save_result(transcript, result)
        st.success("Analysis complete – view results on the Dashboard.")
```

Save Results to SQLite in database.py:

```
import sqlite3, json, datetime

def save_result(transcript: str, result: dict):
    conn = sqlite3.connect("analysis_history.db")
    conn.execute("""CREATE TABLE IF NOT EXISTS history (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        transcript TEXT, score INTEGER, similarity INTEGER,
        grade TEXT, feedback TEXT, created_at TEXT)""")
    conn.execute(
        "INSERT INTO history (transcript, score, similarity, grade, "
        "feedback, created_at) VALUES (?, ?, ?, ?, ?, ?)",
        (transcript, result["score"], result["similarity"],
         result["grade"], json.dumps(result),
         datetime.datetime.now().isoformat()))
    conn.commit()
    conn.close()
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, teacher-friendly interface for VBCUA. This involves designing a dark-themed sidebar layout with a pink accent color for readability and brand consistency, and creating dynamic content rendering to display AI-generated transcripts, scores, similarity, and grades based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base Page Structure:

- Configure each page with `st.set_page_config()`, a sidebar navigation labelled "Choose Page," and a consistent icon-led heading for each section (🏠 Home, 🎤 Upload Audio, 🧠 AI Analysis, 📊 Dashboard, 📄 Report).
- Use Streamlit's native multi-page routing so each file inside `pages/` automatically becomes a sidebar entry.
- Load a shared `assets/style.css` stylesheet for consistent typography, spacing, and the dark navy background with pink headings.

Design a Responsive Layout Using CSS:

```
/* assets/style.css */
section[data-testid="stSidebar"] { background-color: #0d1330; }
h1, h2, h3 { color: #ff3d81; font-weight: 700; }
.stApp { background-color: #0b0f24; color: #e6e6f0; }
.metric-card { background: rgba(255,255,255,0.05); border-radius: 12px;
padding: 1rem; text-align: center; }
```

Create Separate UI Components for Each Output Type:

- **Performance Summary Metrics:** Performance Summary Metrics: three side-by-side metric cards for Score, Similarity, and Grade using `st.columns()` and `st.metric()`.
- **Transcript Panel:** Transcript Panel: a read-only text panel displaying the full Whisper-generated transcript beneath the performance summary.
- **AI Feedback Panel:** AI Feedback Panel: a bulleted "Strengths" and "Gaps" list rendered from the Gemini JSON response on the Report page.

Activity 4.2: Creating Dynamic Content Rendering

Render Results Dynamically:

- Implement a `render_results()` helper that reads `st.session_state["result"]` and `st.session_state["transcript"]` and dynamically populates the Dashboard and Report pages.
- Guard each results page with a session-state check so it gracefully prompts the user to run an analysis first if no result exists yet.

Restore UI State After Generation:

- Use `st.spinner()` during transcription and evaluation, and `st.success()` / `st.error()` banners to confirm completion or surface failures without losing previously entered input.

Ensure Layout Responsiveness and Accessibility:

- Use `st.columns()` with proportional widths so the Performance Summary metrics reflow cleanly on narrower browser windows and tablet screens.

- Apply sufficient colour contrast between the pink headings and the dark navy background to keep transcripts and feedback readable for all users.
- Keep audio playback controls and the Save Input button clearly visible above the fold on the Upload Audio page so instructors can review a recording before committing to an analysis.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the VBCUA application first locally to verify correct operation, and then publicly to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

Activity 5.1: Preparing the Application for Local Deployment

Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Gemini API key. The app loads this automatically using python-dotenv at startup.

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Activity 5.2: Local Testing and Verification

Start the Streamlit Development Server:

```
(myenv) D:\projects>streamlit run Home.py
```

- Once running, Streamlit automatically opens a browser and navigates to "http://localhost:8501" to access the app.
- Interact with the analyser — upload a sample recording, enter an expected answer, run AI Analysis, and verify the transcript, score, similarity, and grade are correctly generated, parsed, and rendered on the Dashboard.
- Download the AI-generated PDF report from the Report page and confirm the transcript, score, similarity, grade, and feedback all appear correctly in the exported file.

Run through the following test cases to validate scoring behaviour across a range of answer quality before sharing the app with real students:

Test Case	Expected Result
Upload a clear, well-articulated recording	High similarity, score above 75, grade A or B
Upload a recording with an incomplete answer	Moderate similarity, score 40–74, grade C or D
Upload a recording unrelated to the expected answer	Low similarity, score below 40, grade F
Submit without an expected answer	Validation error prompting the user to fill in the field
Download the PDF report after analysis	Report opens with transcript, score, similarity, grade, and feedback

Activity 5.3: Public Deployment

Streamlit applications can be shared publicly either through Streamlit Community Cloud or through a tunnelling tool such as Ngrok for quick local sharing.

Deploy via Streamlit Community Cloud:

- Push the project (excluding .env) to a public or private GitHub repository, ensuring requirements.txt lists every dependency.
- Sign in to <https://share.streamlit.io>, select "New app," and point it to the repository and Home.py as the entry file.
- Add GEMINI_API_KEY as a secret under the app's Settings → Secrets so it is available at runtime without being committed to source control.
- Click "Deploy" — Streamlit Community Cloud builds the environment and returns a public *.streamlit.app URL.

Deploy via Ngrok (for quick local sharing):

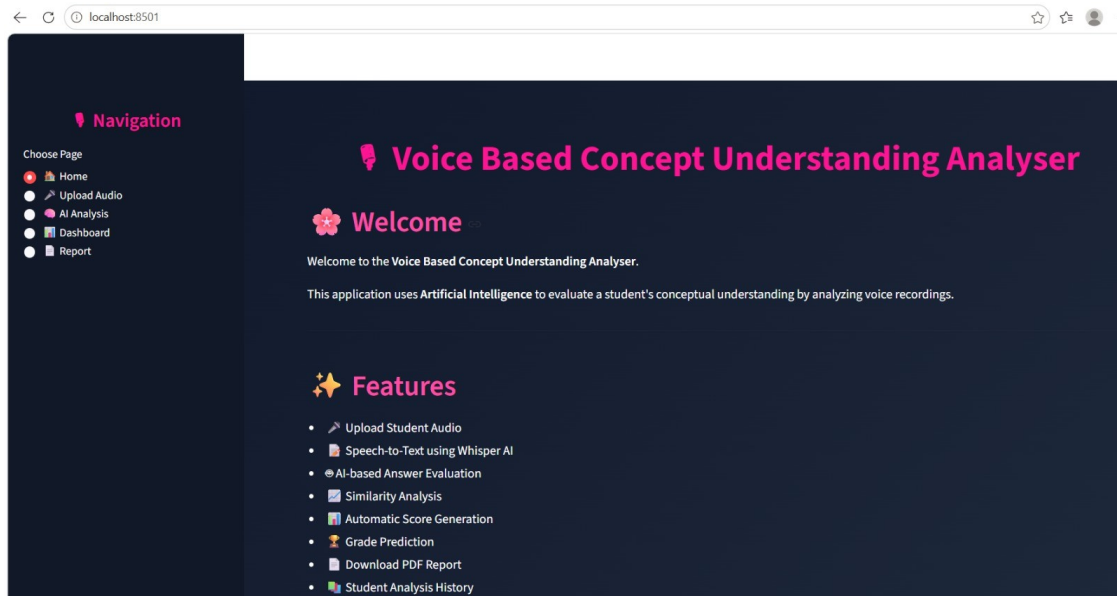
```
D:\projects>pip install pyngrok
D:\projects>python run_public.py
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

Important Notes:

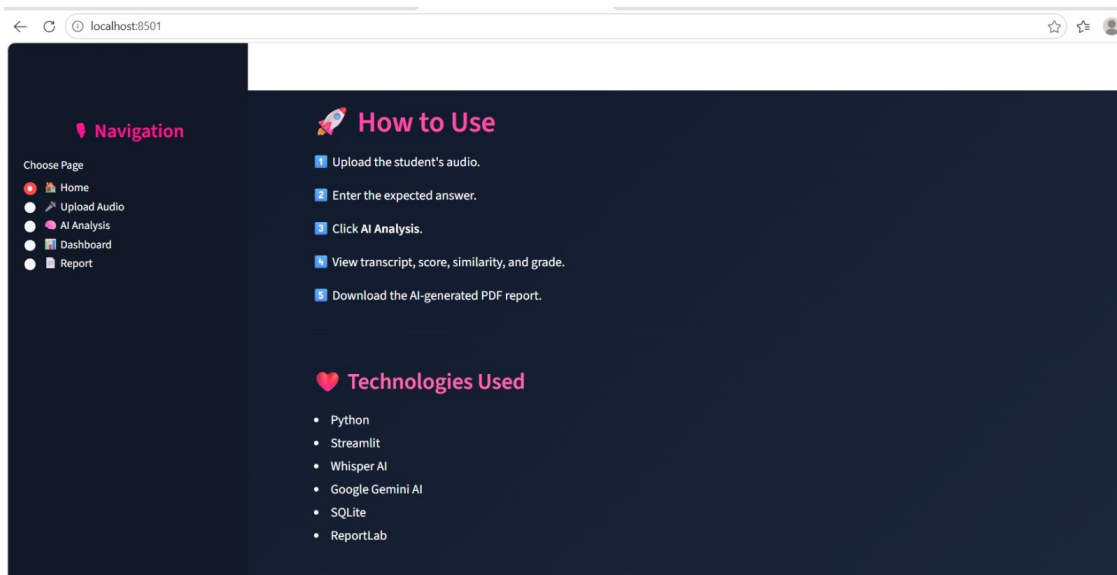
- Whisper's first run downloads the selected model weights locally; ensure sufficient disk space and a stable connection during the first transcription.
- The public Ngrok URL changes each time the tunnel restarts on the free plan. For a persistent URL, use a paid plan or deploy via Streamlit Community Cloud instead.
- Never commit the .env file — store the Gemini API key using Streamlit secrets or environment variables in production.

Exploring the Website's Web Pages:

Home Page:

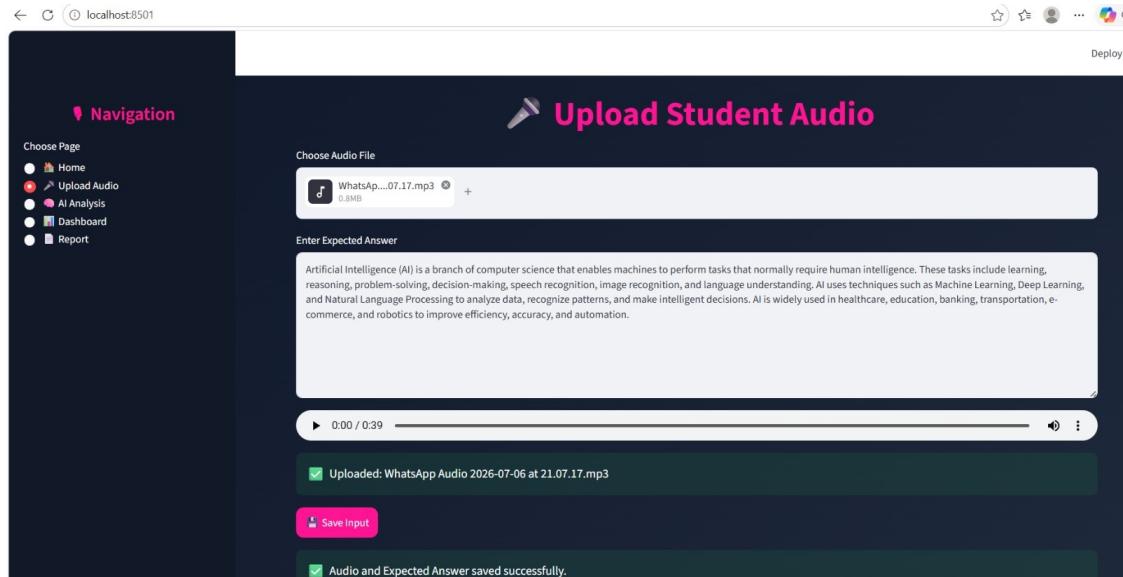


Description: The Home page introduces the Voice Based Concept Understanding Analyser and lists its core capabilities — uploading student audio, Whisper-powered speech-to-text, AI-based answer evaluation, similarity analysis, automatic score generation, grade prediction, downloadable PDF reports, and student analysis history. A pink-accented sidebar labelled "Navigation" lets the user move between Home, Upload Audio, AI Analysis, Dashboard, and Report.



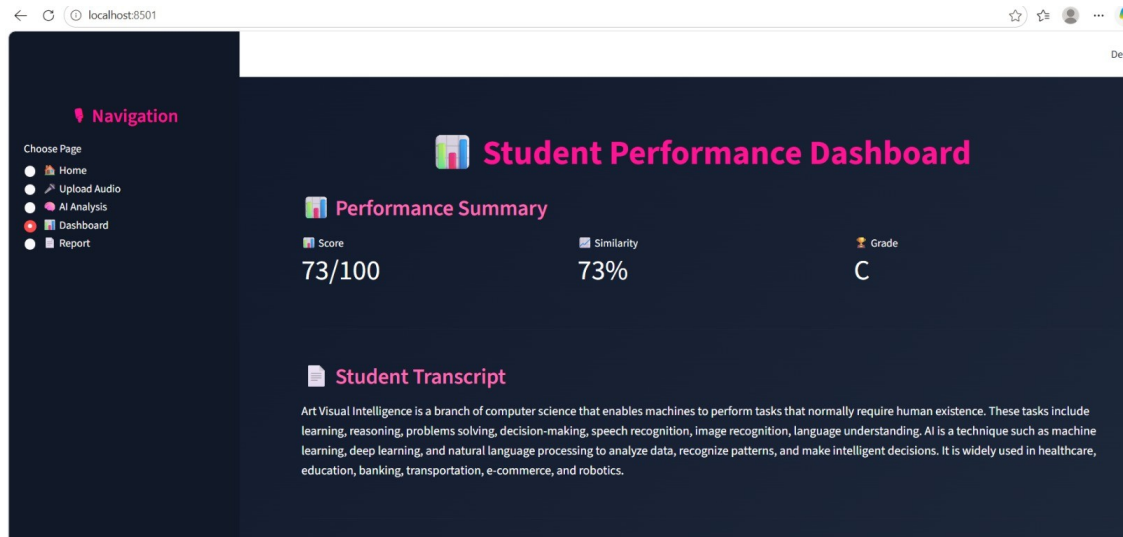
Description: Scrolling further down the Home page reveals the "How to Use" walkthrough — upload the student's audio, enter the expected answer, click AI Analysis, view the transcript/score/similarity/grade, and download the AI-generated PDF report — followed by the "Technologies Used" list: Python, Streamlit, Whisper AI, Google Gemini AI, SQLite, and ReportLab.

Upload Audio Page:



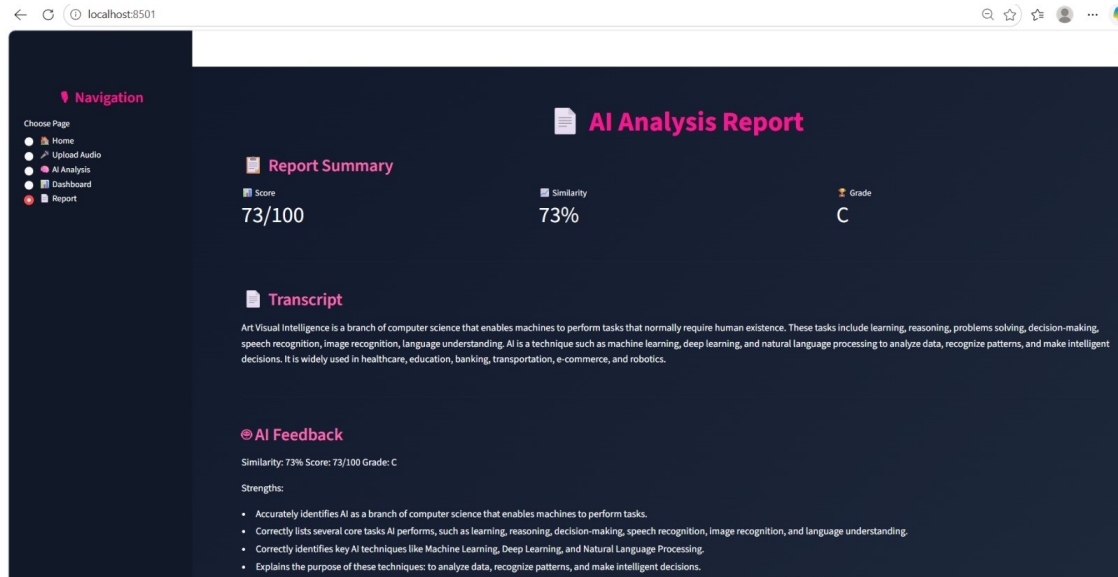
Description: The Upload Audio page contains a file uploader for the student's recording (shown here with a WhatsApp voice note attached), a large text area for entering the expected answer, and a built-in audio player so the teacher can review the clip before analysis. A "Save Input" button persists the audio and expected answer, confirmed by a green success banner reading "Audio and Expected Answer saved successfully."

Dashboard Page:



Description: The Dashboard renders the AI output once analysis is complete. A "Performance Summary" row displays three metrics side by side — Score (73/100), Similarity (73%), and Grade (C) — followed by the full "Student Transcript" produced by Whisper AI from the uploaded recording.

Report Page:



Description: The Report page consolidates the full AI Analysis Report for download. It repeats the Report Summary (Score, Similarity, Grade) and Transcript sections, then presents the "AI Feedback" panel — a structured breakdown starting with the similarity/score/grade line, followed by a bulleted "Strengths" list identifying exactly which parts of the student's spoken answer matched the expected concepts.

Conclusion:

The Voice Based Concept Understanding Analyser is an AI-driven evaluation platform built with Streamlit that streamlines the assessment of spoken conceptual understanding. It uses OpenAI's Whisper AI for accurate speech-to-text transcription and Google's Gemini AI to instantly generate similarity scores, numeric scores, letter grades, and structured feedback for student voice recordings. The project, which included model selection, core feature development, frontend design, and deployment, highlights how targeted AI and a structured framework can boost efficiency for educators evaluating oral responses at scale, with clear potential for future growth.

Looking ahead, VBCUA offers significant opportunities for expansion, such as supporting multiple languages for non-English speaking students, adding speaker-diarization for group discussions, and implementing teacher and student authentication to manage class-wide analysis history securely. By continuously refining the underlying evaluation prompts and enhancing the Dashboard's analytics, the platform is well-positioned to evolve from a specialized grading tool into a comprehensive spoken-assessment assistant for classrooms and training programs of all sizes.